Spring & Spring Boot — Interview + Recall Notes

> **Purpose:** Fast interview revision + notebook-friendly recall.
>
> **Source:** Based on the uploaded `01_springboot_core.pdf`. The notes keep the source's core concepts and flow, with shorter explanations, diagrams, and small code examples.

▼ 1. Client–Server Architecture

Key idea

A **client** requests something. A **server** receives, processes, and responds.

Clients: Browser, mobile app, Postman, React app.

Server: Backend application such as a Spring Boot application.

Recall

```
Client
  |
  | Request
  ▼
Server
  |
  | Response
  ▼
Client
```

Interview:

> Client sends a request; server processes it and sends a response.

▼ 2. HTTP — Language of the Web

HTTP = HyperText Transfer Protocol

HTTP defines the format/rules for communication between client and server.

It defines:

- Request structure
- Response structure
- HTTP method
- URL/path
- Headers
- Data
- Status code

```
Client ── HTTP Request ──▶ Server
Client ◀── HTTP Response ── Server
```

Recall: HTTP is an application-layer protocol working on TCP/IP.

▼ 3. HTTP Request

Main parts

```
HTTP Request
├── Method
├── URL / Path
├── Headers
└── Body
```

Short example

```
POST /login
Content-Type: application/json

{
  "email": "abc@gmail.com",
  "password": "12345"
}
```

Important

- **Method** → what action?
- **Path** → which resource/endpoint?
- **Headers** → additional request information
- **Body** → actual data sent

▼ 4. HTTP Methods ⭐

Method	Purpose	Recall
GET	Read	Fetch data
POST	Create	Create data
PUT	Replace/update completely	Full update
PATCH	Update partially	Partial update
DELETE	Remove	Delete data

```
GET     → READ
POST    → CREATE
PUT     → FULL UPDATE
PATCH   → PARTIAL UPDATE
DELETE  → DELETE
```

Interview: PUT and PATCH are both update operations, but PUT represents complete replacement while PATCH represents partial modification.

▼ 5. HTTP Headers ⭐

Headers are **key-value pairs** carrying additional information.

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer <token>
Host: example.com
```

Must remember

- `Content-Type` → format of request body
- `Accept` → response format client can understand
- `Authorization` → authentication information

Interview connection: Headers are heavily used in JSON APIs, authentication, JWT, and API communication.

▼ 6. HTTP Response ⭐

```
HTTP Response
├── Status Code
├── Headers
└── Body
```

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message":"Login successful"}
```

Important status codes from the notes

Code	Meaning
200	Success
404	Resource not found
500	Internal server error

Recall:

`Request = Method + URL + Headers + Body`

`Response = Status + Headers + Body`

▼ 7. JVM & Java Execution ⭐

Java source is compiled to platform-independent bytecode.



Recall

- `.java` → source code
- `javac` → compiler
- `.class` → bytecode
- JVM → loads and executes bytecode

Interview: This is the basis for Java's **Write Once, Run Anywhere** idea, provided a suitable JVM is available.

▼ 8. Normal Java Program vs Web Application

Normal program

```
Start → Execute → Finish → Exit
```

Web application

```
Start
  ↓
Keep running
  ↓
Wait for request
  ↓
Process request
  ↓
Send response
  ↓
Wait again
```

Key idea: A web application needs to stay alive and continuously handle incoming requests.

▼ 9. Why Core Java Alone Is Not Enough for Web Apps ⭐⭐⭐

Core Java provides:

- Classes
- Objects
- Inheritance
- Collections
- Threads
- Files

But it does not automatically understand web concepts such as:

- HTTP requests
- URLs
- Headers
- Cookies
- Sessions
- REST APIs

For example:

```
GET /hello
```

Core Java does not automatically know that `GET` is an HTTP method and `/hello` is an endpoint.

Problem: Someone must translate HTTP/network data into Java application logic.

▼ 10. Java Networking — The Gap

Java can communicate over networks using `java.net`.

Short example:

```
ServerSocket server = new ServerSocket(8080);
```

But low-level network data is not automatically understood as:

```
GET      → method
/users   → endpoint
Host     → header
```

Without a web technology, developers would need to manually:

1. Open a port
2. Read input streams
3. Parse HTTP
4. Extract method/path/headers/body
5. Route the request
6. Build the response
7. Manage threads/connections
8. Handle errors

Core problem

```
Browser
   | HTTP
   ▼
   ???
   |
   ▼
Java Method
```

This gap led to Servlet technology.

▼ 11. Servlet ⭐⭐⭐

A **Servlet** is a Java object/class designed to handle HTTP requests.

```
HTTP Request
     ↓
  Servlet
     ↓
 Java Code
```

Servlets run inside a **Servlet Container**.

Examples mentioned in the notes:

- Apache Tomcat
- Jetty
- Undertow

Recall

> Servlet = Java-side technology for handling web requests.

▼ 12. Servlet Container ⭐⭐⭐⭐⭐

The Servlet Container sits between the web/network and Java application.

Responsibilities

```
Servlet Container
├── Open port
├── Listen for requests
├── Read TCP bytes
├── Parse HTTP
├── Create request/response objects
├── Manage threads
├── Call servlet methods
├── Send HTTP response
└── Handle connections
```

Instead of manually parsing raw bytes, the container gives Java-friendly request/response handling.

Examples:

- Tomcat
- Jetty
- Undertow

Interview:

> A Servlet Container provides the runtime environment for Servlets and handles low-level web/HTTP work around them.

▼ 13. Why Spring Was Needed ⭐⭐⭐

Servlets solved HTTP handling, but large applications still had problems:

- Too much boilerplate
- Too much configuration
- Tight coupling
- Difficult testing
- Difficult maintenance
- Repeated code

Developers needed better:

- Object management
- Dependency management
- Application organization
- Configuration
- Loose coupling

This is where Spring became important.

▼ 14. Spring Framework ⭐⭐⭐

Spring helps make enterprise Java development **cleaner, easier, and more maintainable**.

Core concepts

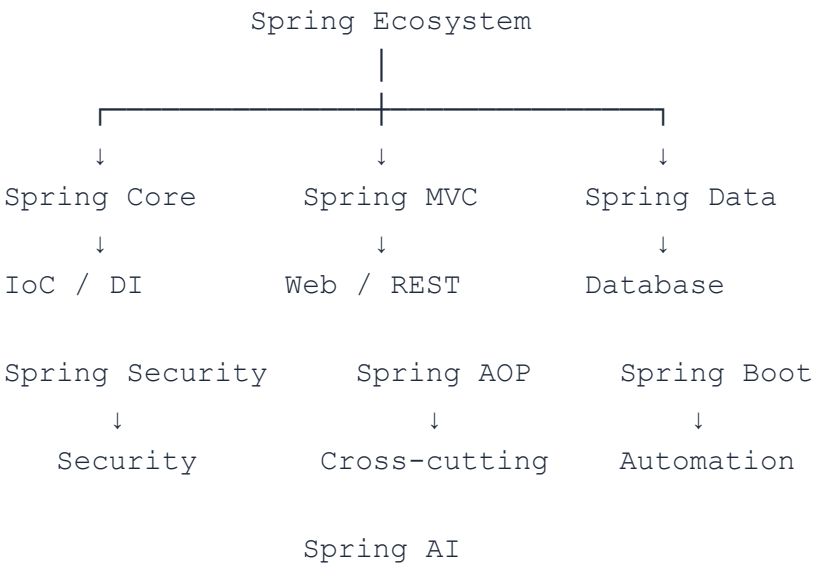
```
Spring
├── IoC
├── Dependency Injection
├── Bean Management
├── Configuration
└── Loose Coupling
```

Interview definition:

> Spring is a Java framework/ecosystem that provides infrastructure for building maintainable applications, including IoC, dependency injection, bean management, and configuration.

▼ 15. Spring Ecosystem ⭐⭐⭐

Spring is an ecosystem, not one small library.



Important areas from the notes:

- Spring Core
- Spring MVC
- Spring Data
- Spring Security
- Spring AOP
- Spring Boot
- Spring AI

▼ 16. Spring Core ⭐⭐⭐⭐⭐

Spring Core is the **foundation** of Spring.

Provides:

- IoC
- Dependency Injection
- Bean management
- Configuration
- `ApplicationContext`

```
Spring Core
    ↓
Foundation
    ↓
Other Spring modules
```

Recall: If you understand Spring Core, many higher-level Spring concepts become easier.

▼ 17. Spring MVC ⭐⭐⭐⭐

Spring MVC is used for:

- Web applications
- REST APIs

Conceptually:

```
Servlet Technology
       +
Spring Core
       ↓
   Spring MVC
```

Instead of directly writing low-level Servlet code:

```
@GetMapping("/hello")
public String sayHello() {
    return "Hello World";
}
```

Important: Spring MVC internally uses Servlet technology but provides a cleaner programming model.

▼ 18. JDBC → Hibernate → JPA → Spring Data JPA ⭐⭐⭐⭐⭐

JDBC

Traditionally, developers had to:

- Write SQL
- Open connections
- Execute queries
- Handle result sets
- Close resources

ORM

Hibernate maps:

```
Java Object  ↔  Database Table
```

ORM = **Object-Relational Mapping**

JPA vs Hibernate

JPA	Hibernate
Specification	Implementation
Defines rules/contracts	Implements persistence behavior

Recall:

> JPA tells **what** should be done; Hibernate provides an implementation.

Spring Data JPA flow

```
Repository
    ↓
Spring Data JPA
    ↓
Hibernate
    ↓
JDBC
    ↓
Database
```

Interview: Spring Data JPA reduces persistence boilerplate and works with JPA implementations such as Hibernate.

▼ 19. Spring Security ⭐⭐⭐⭐

Used for:

- Authentication
- Authorization
- JWT
- OAuth
- Roles
- Permissions
- Password encoding
- CSRF protection
- Access control

Authentication vs Authorization

```
Authentication
      ↓
Who are you?

Authorization
      ↓
What are you allowed to do?
```

Example:

```
USER  → View profile
ADMIN → Delete user
```

Recall: Authentication = identity. Authorization = permissions/access.

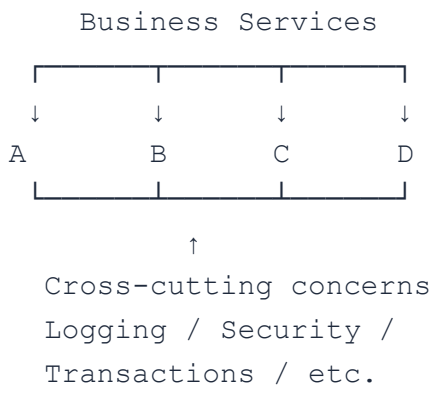
▼ 20. Spring AOP ⭐⭐⭐

AOP = Aspect-Oriented Programming

Used to separate **cross-cutting concerns** from business logic.

Examples:

- Logging
- Security checks
- Transaction management
- Performance tracking
- Exception handling



Recall: Cross-cutting = functionality that affects many parts of an application.

▼ 21. Spring Boot ⭐⭐⭐⭐⭐

Spring Boot is NOT a replacement for Spring.

It is an **automation layer on top of Spring**.

Main benefits

```
Spring Boot
├── Auto-configuration
├── Starter dependencies
├── Embedded servers
├── Sensible defaults
├── Production-ready features
└── Less manual configuration
```

Core idea

> Spring provides the framework capabilities; Spring Boot makes setup and configuration much easier.

▼ 22. Spring Boot — Opinionated Framework ⭐⭐⭐⭐

Opinionated means Spring Boot provides sensible defaults and makes assumptions to reduce configuration.

For example, with web dependencies it can configure/support:

- Embedded Tomcat
- Spring MVC setup
- JSON support
- Basic error handling
- Default configuration

```
Developer adds dependency
          ↓
Spring Boot detects setup
          ↓
Applies sensible defaults
          ↓
Less manual configuration
```

Interview definition:

> An opinionated framework provides sensible defaults so developers do not have to configure everything manually.

▼ 23. Spring vs Spring Boot ⭐⭐⭐⭐⭐

Spring Framework	Spring Boot
Provides core framework/modules	Simplifies Spring setup
More manual configuration	Less manual configuration
Foundation of ecosystem	Built on Spring
Flexible	Sensible defaults
Developer configures more	Auto-configuration reduces setup

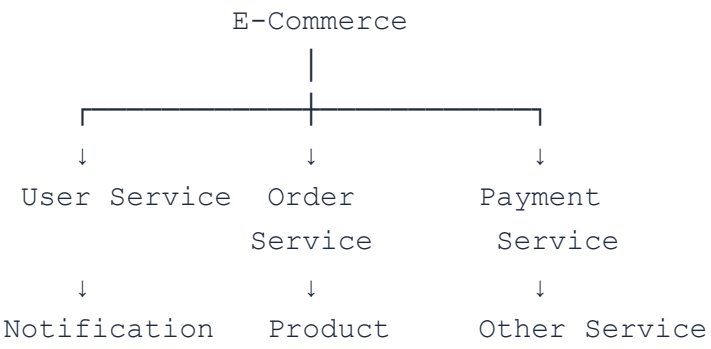
One-line interview answer

> **Spring Boot uses Spring; it does not replace Spring.**

▼ 24. Microservices ⭐⭐⭐⭐

Microservices are an **architecture style**, not a Spring module.

A large application can be split into smaller independent services.



Services can be independently:

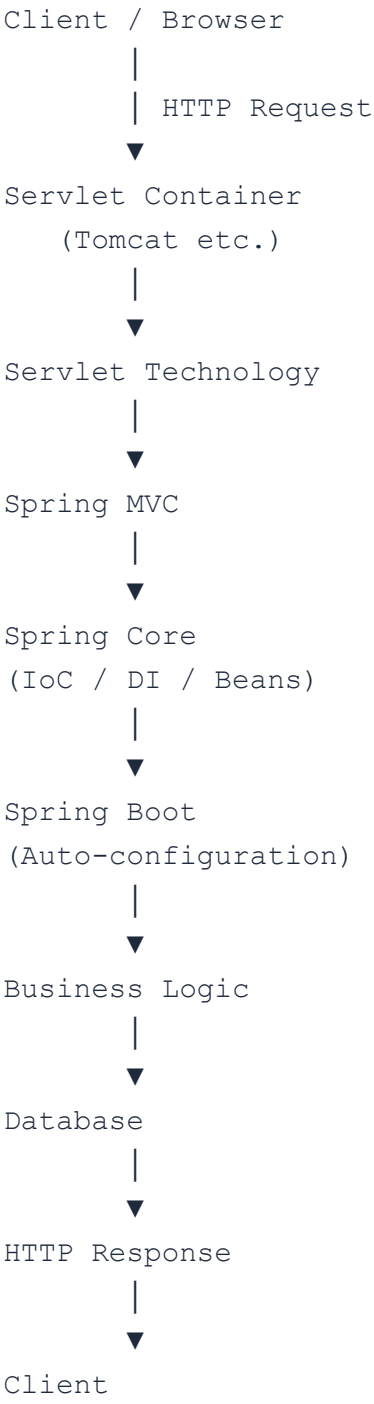
- Developed
- Deployed
- Scaled

Recall

```
Microservices = Architecture style
Spring Boot   = Common tool for building services
```

▼ 25. Complete Request Flow ⭐⭐⭐⭐⭐

Browser → Spring Boot



Why this matters

When you write:

```
@RestController
@GetMapping("/hello")
```

you should understand that multiple layers are working behind the scenes.

Recall chain:

> HTTP → Servlet Container → Servlet → Spring MVC → Spring Core → Spring Boot → Business Logic

---

▼ 26. Complete Evolution ⭐⭐⭐⭐⭐

```
Client-Server Architecture
          ↓
HTTP Communication
          ↓
Core Java limitation for Web
          ↓
Java Networking / Sockets
          ↓
Manual HTTP Parsing Problem
          ↓
Servlets + Servlet Container
          ↓
Spring Framework
          ↓
Spring Ecosystem
          ↓
Spring Boot
          ↓
Modern Java Backend
```

The "WHY" story

```
HTTP
 ↓
How does Java handle it?
 ↓
Servlets
 ↓
How do we make enterprise apps maintainable?
 ↓
Spring
 ↓
How do we reduce Spring setup/configuration?
 ↓
Spring Boot
```

▼ 27. ⭐ Rapid Interview Recall

HTTP

Q: What is HTTP?
A: Application-layer protocol used for client-server communication.

Servlet

Q: What is a Servlet?
A: Java technology/object used to handle HTTP requests.

Servlet Container

Q: What does Tomcat do?
A: Provides the Servlet runtime and handles low-level web/HTTP work around Servlets.

Spring

Q: Why Spring?
A: To make enterprise Java development cleaner, maintainable, loosely coupled, and easier to manage.

Spring Core

Q: What does Spring Core provide?
A: IoC, DI, bean management, configuration, and `ApplicationContext`.

Spring MVC

Q: What is Spring MVC?
A: Spring's web framework for web applications and REST APIs.

JPA

Q: Is JPA an implementation?
A: No. JPA is a specification.

Hibernate

Q: What is Hibernate?
A: A popular implementation of JPA.

Spring Boot

Q: What is Spring Boot?
A: An automation layer on Spring that reduces configuration and speeds up application setup.

Opinionated

Q: What does opinionated mean?
A: Provides sensible defaults.

Microservices

Q: Is microservices a Spring module?
A: No. It is an architecture style.

▼ 28. ⭐ Must-Draw Diagrams Before Interview

Diagram 1 — Request/Response

```
Client ── Request ──▶ Server
Client ◀── Response ── Server
```

Diagram 2 — Spring Boot Request Flow

```
Client
  ↓
HTTP
  ↓
Tomcat / Servlet Container
  ↓
Servlet
  ↓
Spring MVC
  ↓
Spring Core
  ↓
Business Logic
  ↓
Database
```

Diagram 3 — Persistence

```
Spring Data JPA
       ↓
   Hibernate
       ↓
      JDBC
       ↓
   Database
```

Diagram 4 — Technology Evolution

```
HTTP
 ↓
Sockets
 ↓
Servlets
 ↓
Spring
 ↓
Spring Boot
```

Final 60-Second Recall

```
CLIENT
  ↓
HTTP REQUEST
  ↓
SERVLET CONTAINER / TOMCAT
  ↓
SERVLET
  ↓
SPRING MVC
  ↓
SPRING CORE
(IoC / DI / Beans)
  ↓
SPRING BOOT
(Auto Configuration)
  ↓
BUSINESS LOGIC
  ↓
SPRING DATA JPA
  ↓
HIBERNATE
  ↓
JDBC
  ↓
DATABASE
  ↓
HTTP RESPONSE
  ↓
CLIENT
```

Remember these 10 lines

1. **HTTP = communication protocol between client and server.**
2. **Core Java does not automatically understand HTTP/web requests.**
3. **Servlets brought standard Java web request handling.**
4. **Servlet Container manages Servlets and low-level web work.**
5. **Spring was introduced to simplify enterprise Java development.**
6. **Spring Core provides IoC, DI and bean management.**
7. **Spring MVC simplifies web/REST development.**

8. **JPA is a specification; Hibernate is an implementation.**
9. **Spring Boot automates/configures Spring and provides sensible defaults.**
10. **Microservices are an architecture style, not a Spring module.**